


**Sukkur Institute of Business Administration University**
Department of Computer Science

**CS304 – Software Quality Engineering**
BS – V (Software Engineering) – "A"
Fall 2026

**Instructor:** Dr. Khursheed Ali

# Bookstore Management System

## Software Design Specification

## Submission Profile

| | |
|---|---|
| **Name:** Muhammad Anas Qadri | **CMS ID:** 053-24-0016 |
| **Name:** Isra Memon | **CMS ID:** 053-24-0017 |
| **Name:** Muhammad Waleed | **CMS ID:** 053-24-0100 |

# Table of Contents

1. System Architecture ..... 3
1.1. Architecture Overview ..... 3
1.2. Components ..... 4
1.3. How the Design Meets Key Requirements ..... 4
2. Database Design ..... 5
2.1. ER Diagram ..... 5
2.2. Data Dictionary ..... 6
2.3. Relationships and Rules ..... 6
3. User Interface Prototypes ..... 7
3.1. Screen List ..... 7
3.2. Prototypes ..... 7
4. GitHub Repository ..... 8
4.1. Repository Details ..... 8
4.2. Repository Structure ..... 8
4.3. Branching, Commit and Review Rules ..... 9
4.4. Issue Tracking ..... 9
4.5. Screenshots ..... 9

# 1. System Architecture

## 1.1. Architecture Overview

BMS is a desktop program that runs fully on the shop's computer. It has three main parts. The Electron app opens the program window. Inside it, the Next.js screens (the user interface) talk to the Spring Boot backend through a REST API. The backend saves and reads data in an SQLite database file on the same computer. Nothing needs the internet. Figure 1 shows the parts.

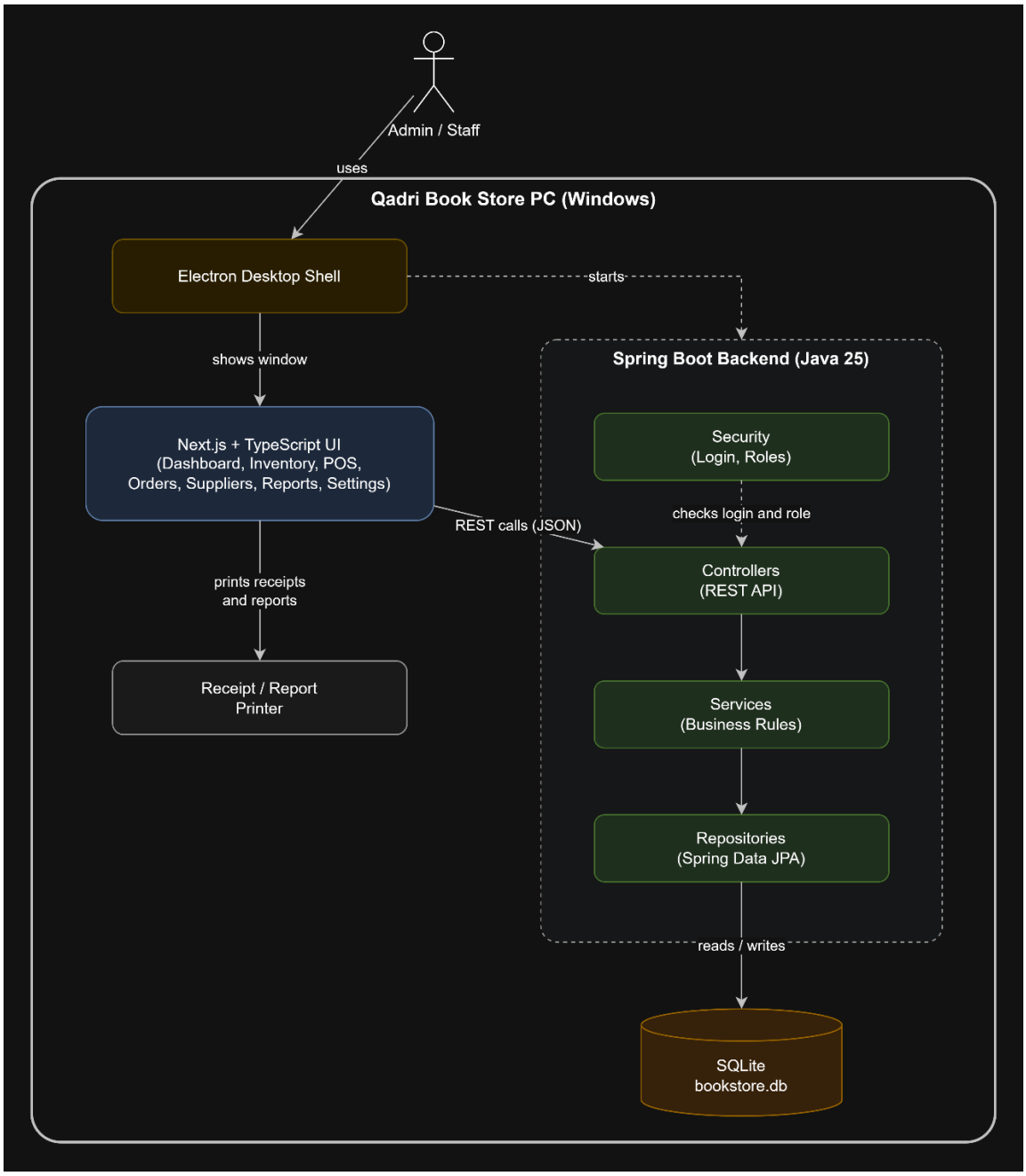



*Figure 1: System Architecture (Component Diagram)*

**Example:** Staff clicks "Generate Bill". The screen sends the sale to the backend. The backend checks the data, saves the sale, lowers the stock and returns the bill. The screen shows the bill.

## 1.2. Components

| Component | Technology | What it does |
|---|---|---|
| **Desktop Shell** | Electron | Opens the program window and starts the backend. |
| **User Interface** | Next.js 16, React 19, TypeScript, Tailwind CSS | All screens: dashboard, inventory, sales, orders, suppliers, reports, settings. |
| **Backend API** | Spring Boot 4.1.2, Java 25 | Business rules, checks, calculations and REST endpoints. |
| **Security** | Spring Security | Login, roles (Admin or Staff) and password hashing. |
| **Data Access** | Spring Data JPA, Hibernate | Reads and writes database tables as Java objects. |
| **Database** | SQLite (data/bookstore.db) | Stores all shop data on the local PC. |

The backend code is split into four layers, so each part has one job:

| Layer | Job | Example |
|---|---|---|
| **Controller** | Receives requests from the screens and sends back answers. | Receive a new sale |
| **Service** | Holds the business rules. | Add up the total, lower the stock |
| **Repository** | Talks to the database. | Save a sale, find a product |
| **Entity** | A table written as a Java class. | Product, Sale, User |

## 1.3. How the Design Meets Key Requirements

- **Works offline (NFR-09):** everything runs on the shop PC and the database is a local file.
- **Login and roles (NFR-10, NFR-11):** Spring Security checks the login. Passwords are saved hashed. The backend checks the role on every request, so Staff cannot reach Admin features by another way.
- **Correct data (NFR-04, NFR-06):** the screen checks the input first and the backend checks it again before saving. A sale is saved all-or-nothing (items, total and stock change together).
- **Backup and restore (NFR-08, FR-25, FR-26):** the whole database is one file, so backup copies that file (for example to a USB drive) and restore puts it back.
- **Activity log (NFR-12):** the backend writes login, sale and delete actions into the activity log table.

# 2. Database Design

## 2.1. ER Diagram

The database has 11 tables. Figure 2 shows the tables and how they connect.

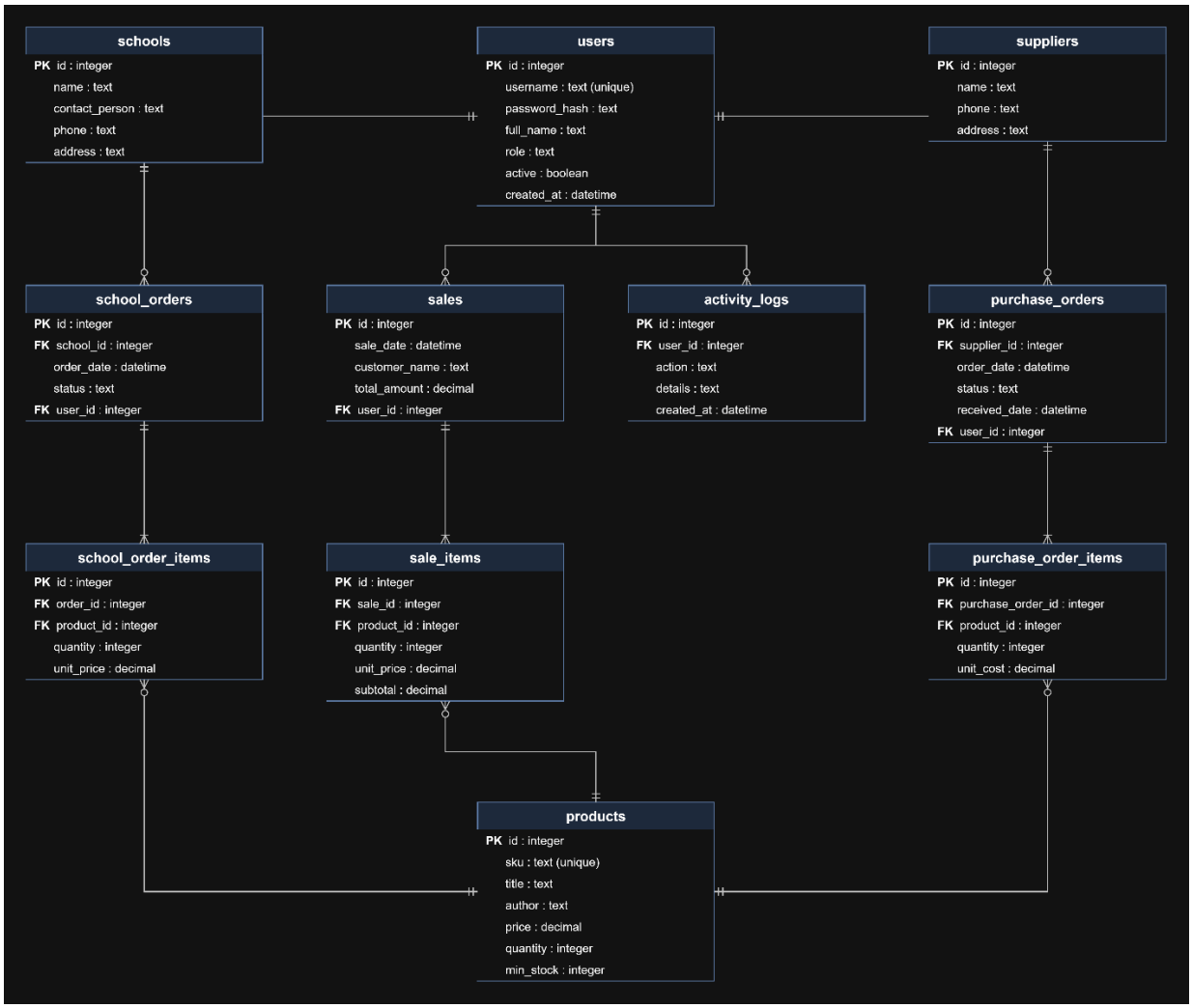



*Figure 2: Entity Relationship (ER) Diagram*

## 2.2. Data Dictionary

**PK** = primary key (a unique id for each row). **FK** = foreign key (a link to a row in another table).

| Table | Columns | Purpose |
|---|---|---|
| **users** | **id** (PK), username (unique), password_hash, full_name, role (ADMIN or STAFF), active, created_at | Login accounts (FR-01, FR-02, FR-24) |
| **products** | **id** (PK), sku (unique), title, author (optional), price, quantity, min_stock | Books and stationery items (FR-04 to FR-09) |
| **sales** | **id** (PK), sale_date, customer_name (optional), total_amount, user_id (FK) | One row for each sale (FR-10, FR-12, FR-14) |
| **sale_items** | **id** (PK), sale_id (FK), product_id (FK), quantity, unit_price, subtotal | Items inside a sale (FR-11, FR-13) |
| **schools** | **id** (PK), name, contact_person, phone, address | Registered schools (FR-15) |
| **school_orders** | **id** (PK), school_id (FK), order_date, status, user_id (FK) | Bulk orders and their status (FR-16, FR-18) |
| **school_order_items** | **id** (PK), order_id (FK), product_id (FK), quantity, unit_price | Items inside a school order. The quotation is printed from these (FR-17) |
| **suppliers** | **id** (PK), name, phone, address | Supplier records (FR-19) |
| **purchase_orders** | **id** (PK), supplier_id (FK), order_date, status (ORDERED or RECEIVED), received_date, user_id (FK) | Orders sent to suppliers (FR-20, FR-21) |
| **purchase_order_items** | **id** (PK), purchase_order_id (FK), product_id (FK), quantity, unit_cost | Items inside a purchase order |
| **activity_logs** | **id** (PK), user_id (FK), action, details, created_at | Log of login, sale and delete actions (NFR-12) |

## 2.3. Relationships and Rules

**Relationships**

- One user makes many sales, school orders, purchase orders and log entries.
- One sale has many sale items. Each item is for one product.
- One school has many school orders. Each order has many items.
- One supplier has many purchase orders. Each order has many items.

**Rules**

- The sku must be unique and not empty (FR-04).
- The price must be more than 0. The quantity and min_stock cannot be negative (NFR-06).
- A product is low on stock when quantity is equal to or below min_stock (FR-09).
- Item rows keep the price used at that time, so old bills do not change if the price changes later.
- A product that is already used in a sale or order cannot be deleted. This keeps old records correct.
- A school order status is PENDING, READY or DELIVERED (FR-18). Marking a purchase order RECEIVED adds its quantities to stock (FR-21).
- The backup is a copy of the file bookstore.db (FR-25).

# 3. User Interface Prototypes

The program has one window. A dark menu is on the left and the working area is on the right. The menu has seven items: Dashboard, Books Inventory, Sales / POS, Suppliers, School Orders, Reports and Settings. All screens use the same layout. Deleting asks for confirmation and every action shows a message (NFR-13, NFR-14).

## 3.1. Screen List

| Screen | What it shows | Requirements |
|---|---|---|
| **Login** | Username, password and a Login button. A wrong login shows an error. | FR-01 to FR-03 |
| **Dashboard** | Cards for Total Books, Today's Sales, Low Stock and Suppliers. A Recent Sales table. | FR-09, FR-22 |
| **Books Inventory** | Book list, search, Add / Edit form and Delete with "Are you sure?". | FR-04 to FR-08 |
| **Sales / POS** | Choose a book and quantity, a cart with subtotals, the total, Generate Bill, a bill preview and New Sale. | FR-10 to FR-14 |
| **School Orders** | Register a school, create an order, make a quotation and update the status. | FR-15 to FR-18 |
| **Suppliers** | Supplier list, purchase orders and goods received. | FR-19 to FR-21 |
| **Reports** | Sales report by date, inventory report and a Print button. | FR-22, FR-23, FR-27 |
| **Settings** | User accounts, Backup and Restore. | FR-24 to FR-26 |

## 3.2. Prototypes

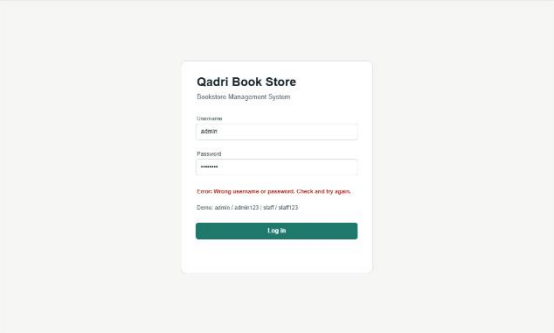



*Figure 3: Login*

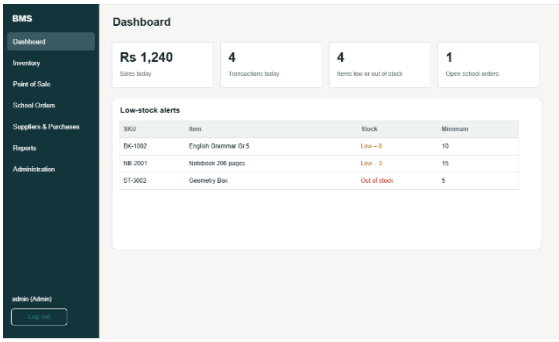



*Figure 4: Dashboard*

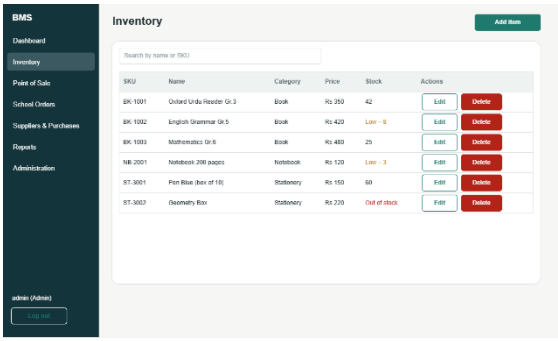



*Figure 5: Inventory*

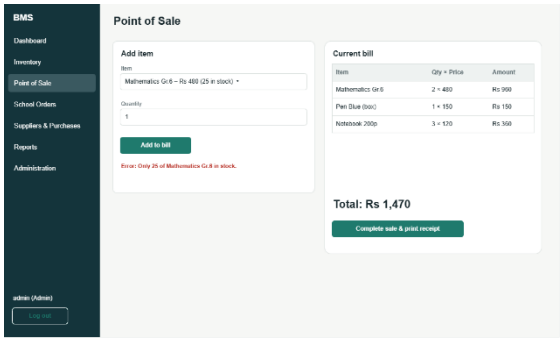



*Figure 6: Point of Sale*

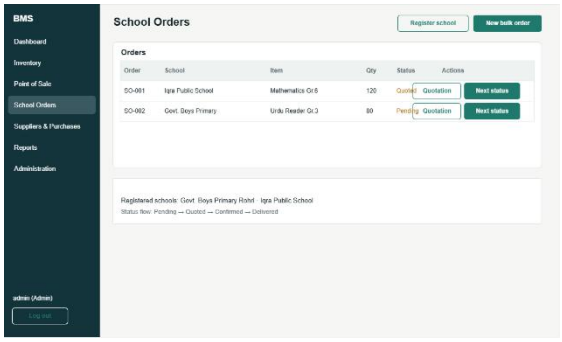



*Figure 7: School Order*

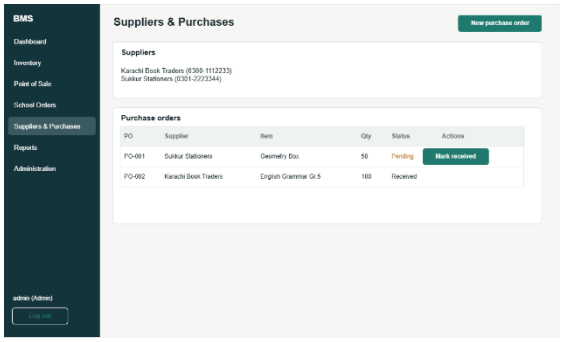



*Figure 8: Suppliers*

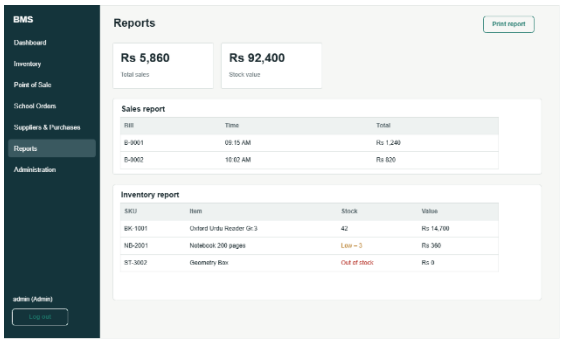



*Figure 9: Reports*

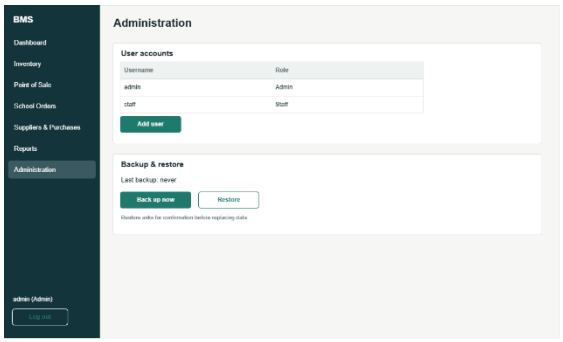



*Figure 10: Administration Settings*

# 4. GitHub Repository

## 4.1. Repository Details

| Item | Detail |
|---|---|
| **Link** | https://github.com/anasqadri-dev/bookstore-management-system |
| **Visibility** | Public |
| **License** | MIT |
| **Main branches** | main (stable code) and develop (where work is joined) |
| **Contains** | Backend and frontend source code, project documents, issue templates, a pull request template, README and CONTRIBUTING files |

## 4.2. Repository Structure

| Folder / File | Contents |
|---|---|
| **backend-bookstore-management-system/** | Spring Boot backend (Maven, JPA, Security, Validation, SQLite setup). |
| **frontend/** | Next.js app with the Dashboard, Books Inventory and Sales / POS screens. |
| **docs/** | Project documents (Project Proposal and SQE Project Requirements). The SRS and this SDS will be added. |
| **.github/** | Issue templates (bug, feature, task) and the pull request template. |
| **README.md** | Project overview, problem, features, technology stack, architecture, quality goals, team and schedule. |
| **CONTRIBUTING.md** | Branch, commit, testing and review rules. |
| **LICENSE** | MIT license. |

## 4.3. Branching, Commit and Review Rules

| Rule | Detail |
|---|---|
| **Branches** | main, develop, feature/*, bugfix/*, test/*, docs/* |
| **Commit messages** | Start with a prefix: feat, fix, test, docs, refactor, style or chore. Example: feat(frontend): add sales POS interface |
| **Work steps** | Pick an issue, create a branch from develop, commit, open a pull request to develop, get a review from another member, then merge. |
| **Pull requests** | Each one names the issue it closes, the requirement it covers, the tests done and screenshots for UI changes. |

## 4.4. Issue Tracking

Every task starts as a GitHub issue made from one of three templates. A pull request then closes the issue.

| Template | Label | Main sections |
|---|---|---|
| **Bug Report** | type: bug | Description, environment, preconditions, steps to reproduce, expected and actual result, severity, evidence, related requirement |
| **Feature Request** | type: feature | Proposed feature and its related requirement |
| **General Task** | type: chore | Objective, background, scope, expected deliverables |

## 4.5. Screenshots

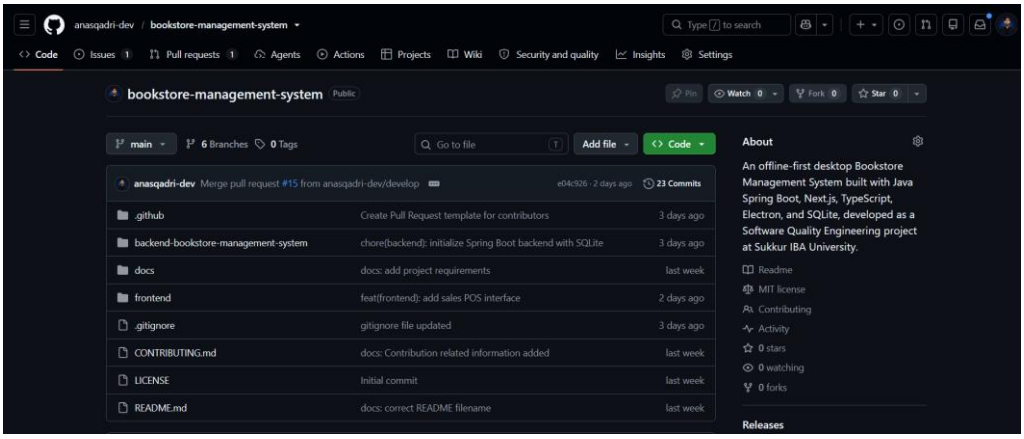



*Figure 11: Repository Home Page*

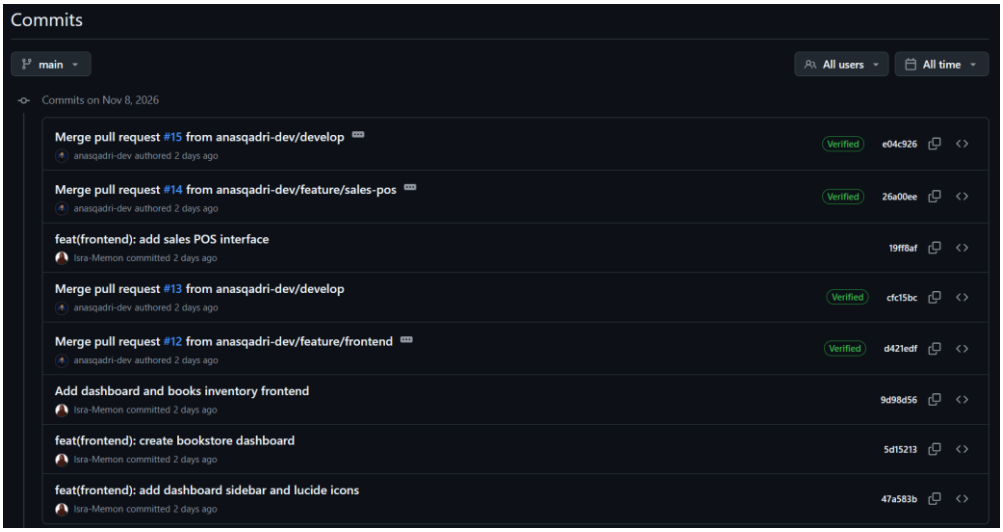



*Figure 12: Commit History*

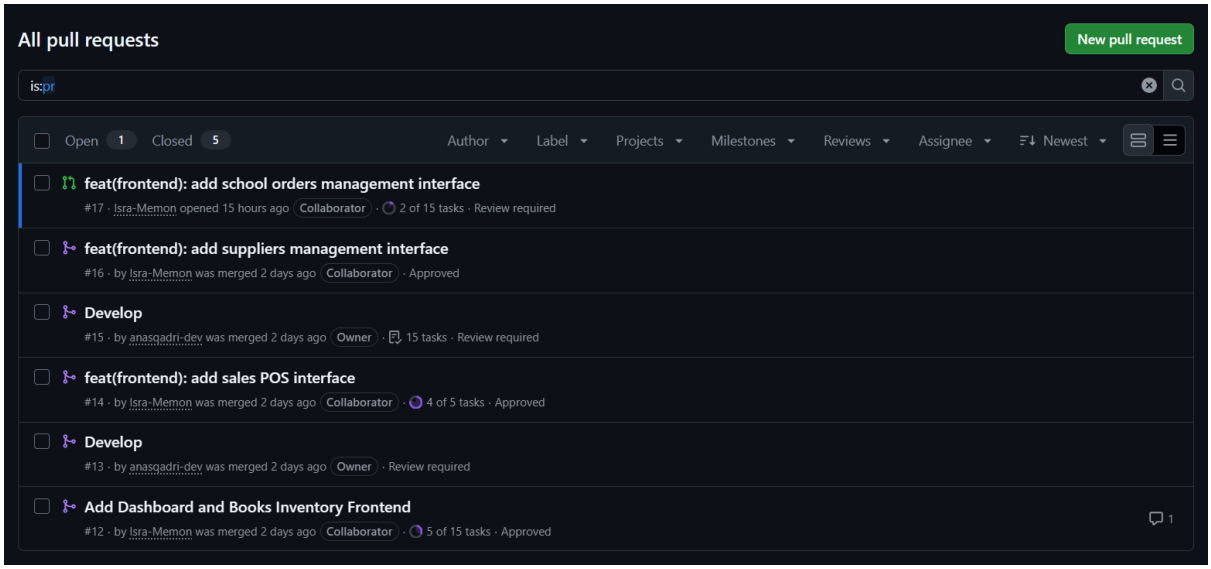



*Figure 13: Pull Requests*

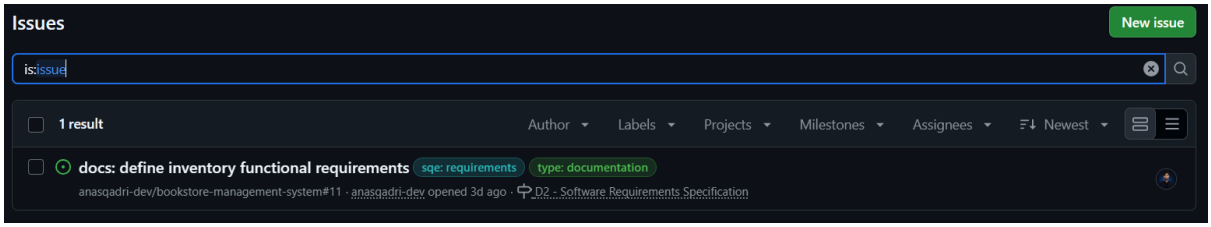



*Figure 14: Issues*

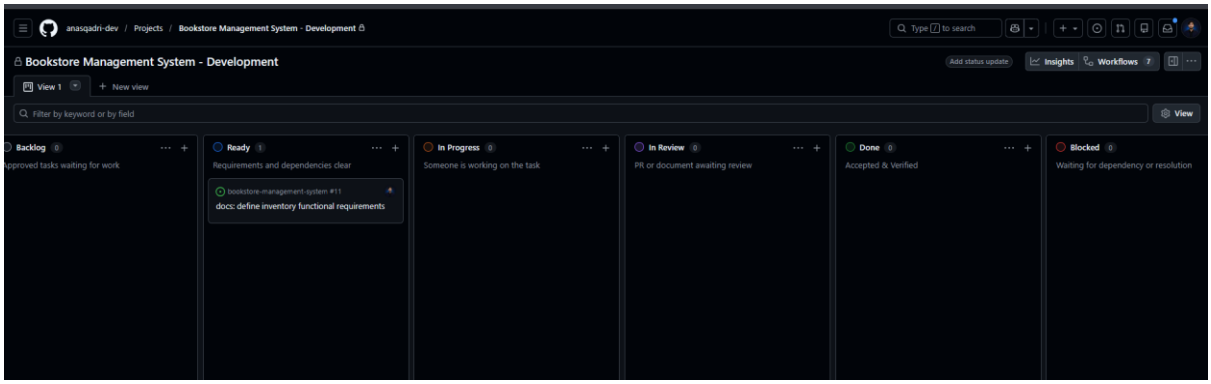



*Figure 15: Project Board*